

Handover Report — De Erfeniswijzer

1. Project Overview

"De Erfeniswijzer" ("The Legacy Guide") is a Dutch website about inheritance and estate planning ("nalatenschap en erfenis"). The project has two language variants:

- A Dutch (NL) site, which is the default build.
- An English (EN) site, built as a separate deployment.

Both variants are built from the same codebase; the language is selected at build time (see section 6) and can also be toggled by the visitor in the browser (see section 8).

2. Tech Stack

- **Framework:** [TanStack Start](#) — a full-stack/SSR meta-framework built on Vite and Nitro (comparable to Next.js or Remix). Confirmed by the dependencies `@tanstack/react-start`, `@tanstack/react-router`, and `@tanstack/router-plugin` in `package.json`.
- **Rendering:** Server-side rendering (SSR) via Nitro, targeting Cloudflare Workers. The build produces a server bundle (`dist/server/server.js`) in addition to static client assets — this is not a static-only site.
- **UI library:** React 19.2.0 / React DOM 19.2.0.
- **Styling:** Tailwind CSS v4 (via `@tailwindcss/vite`), with `class-variance-authority`, `clsx`, and `tailwind-merge` for class composition.
- **UI components:** shadcn/ui-style components in `src/components/ui/` (~46 files), built on top of Radix UI primitives (`@radix-ui/react-*` packages) — accordion, dialog, dropdown-menu, tabs, tooltip, select, popover, navigation-menu, etc.
- **Forms/validation:** `react-hook-form`, `@hookform/resolvers`, `zod`.
- **Other notable UI dependencies:** `lucide-react` (icons), `sonner` (toast notifications), `cmdk` (command palette), `embla-carousel-react` (carousels), `vaul` (drawers), `recharts` (charts), `react-day-picker`, `date-fns`.
- **Language:** TypeScript, `tsconfig.json` set to `target: ES2022`, `module: ESNext`, bundler module resolution, strict mode enabled, with a `@/*` path alias mapped to `./src/*`.
- **Build tool:** Vite 8.0.16, using a shared preset package `@lovable.dev/vite-tanstack-config` that wraps the TanStack Start Vite plugin, `@vitejs/plugin-react`, Tailwind, `vite-tsconfig-paths`, Nitro (Cloudflare target), and some dev-only tooling.
- **Origin/tooling note:** This project was built and edited using [Lovable.dev](#), an AI-assisted app builder. This is evidenced by the `@lovable.dev/vite-tanstack-config` dependency, a `.lovable/plan.md` file containing Lovable's internal build/design plan, and a `src/lib/lovable-error-reporting.ts` file that reports errors to Lovable's editor iframe (a no-op outside of the Lovable environment).
- **Node.js version:** No `.nvmrc` file and no `engines` field are defined in `package.json`. The presence of `"@types/node": "^22.16.5"` in `devDependencies` implies Node 22 was used during development, but this is not explicitly pinned anywhere in the repo.

3. Repository Structure

```
erfenis-rust-gids/
■■■ src/
■   ■■■ routes/                # TanStack Router file-based routes (pages)
■   ■   ■■■ __root.tsx        # root layout: <html>, <head>, Header/Footer, providers
■   ■   ■■■ index.tsx         # homepage
■   ■   ■■■ hulp-bij-erfenis.tsx
■   ■   ■■■ bij-leven-regelen.tsx
■   ■   ■■■ kennisbank.tsx    # knowledge base / articles listing
■   ■   ■■■ gratis-gids.tsx    # "free guide" page
■   ■   ■■■ contact.tsx
■   ■   ■■■ over-ons.tsx       # about us
■   ■   ■■■ bedankt.tsx        # thank-you page
■   ■■■ components/           # page-level components (Header, Footer, PageHeader, ContentHero,
    CtaSection, Reveal)
■   ■   ■■■ ui/                # shadcn/ui-style primitive components (Radix UI wrappers)
■   ■■■ lib/
■   ■   ■■■ i18n.tsx           # language context/provider (NL/EN)
■   ■   ■■■ translations.ts    # translation/content object (~1150 lines) for both languages
■   ■   ■■■ config.server.ts   # server-only env accessor
■   ■   ■■■ error-capture.ts, error-page.ts, lovable-error-reporting.ts # SSR error handling /
    Lovable telemetry hook
■   ■   ■■■ utils.ts
■   ■   ■■■ api/example.functions.ts # boilerplate TanStack Start server-function example
■   ■■■ assets/                # images (hero, team, testimonials, logo), imported directly by
    components
■   ■■■ router.tsx, routeTree.gen.ts # TanStack Router setup (routeTree.gen.ts is auto-generat
    ed)
■   ■■■ server.ts              # Cloudflare Worker fetch handler wrapping the TanStack Start SS
    R entry
■   ■■■ start.ts               # TanStack Start app entry
■   ■■■ styles.css             # global Tailwind CSS
■■■ dist/                      # build output (gitignored): dist/client (static assets + _work
    er.js), dist/server (server.js)
■■■ wrangler.toml              # Cloudflare deployment config for the NL site
■■■ wrangler.en.toml          # Cloudflare deployment config for the EN site
■■■ vite.config.ts
■■■ components.json            # shadcn/ui config
■■■ tsconfig.json, eslint.config.js, .prettierrc
■■■ bun.lock, package-lock.json, bunfig.toml
■■■ .lovable/, .tanstack/      # Lovable.dev platform metadata / TanStack tooling
```

There is no `public/` directory — all images live under `src/assets/` and are imported as ES modules directly in components, so Vite bundles and hashes them at build time.

4. Build & Run Commands

Scripts defined in `package.json`:

```
"scripts": {
  "dev": "vite dev",
  "build": "vite build",
  "build:pages": "vite build && node -e \"require('fs').copyFileSync('dist/server/server.js','dist/client/_worker.js')\"",
  "build:pages:en": "cross-env VITE_DEFAULT_LANG=en vite build && node -e \"require('fs').copyFileSync('dist/server/server.js','dist/client/_worker.js')\"",
  "build:dev": "vite build --mode development",
  "preview": "vite preview",
```

```

"lint": "eslint .",
"format": "prettier --write ."
}

```

- `dev` — starts the Vite dev server with TanStack Start SSR and hot module reloading, for local development.
- `build` — production build. Produces `dist/client` (static assets) and `dist/server/server.js` (SSR server bundle, built for the Cloudflare Workers Nitro target).
- `build:pages` — runs `vite build`, then copies `dist/server/server.js` to `dist/client/_worker.js`. Cloudflare Pages auto-detects `_worker.js` in the output directory as the SSR handler. This produces the Dutch (NL) build, using the default language.
- `build:pages:en` — identical to `build:pages`, but sets the `VITE_DEFAULT_LANG=en` environment variable (via `cross-env`) before building, producing an English-default build for the separate EN deployment.
- `build:dev` — builds using Vite's "development" mode (unminified) instead of the production build path.
- `preview` — serves the built output locally via `vite preview`.
- `lint` — runs ESLint across the project (config in `eslint.config.js`, using `typescript-eslint`, `eslint-plugin-react-hooks`, `eslint-plugin-react-refresh`, and Prettier integration).
- `format` — runs Prettier `--write` across the repo (rules in `.prettierrc`, exclusions in `.prettierignore`).

5. Deployment Configuration

The project deploys to **Cloudflare Workers/Pages** as two separate deployments — one per language.

`wrangler.toml` (NL site):

```

name = "erfenis-rust-gids"
main = "dist/server/server.js"
compatibility_date = "2024-09-23"
compatibility_flags = ["nodejs_compat"]

```

```

[assets]
directory = "./dist/client"

```

`wrangler.en.toml` (EN site):

```

name = "erfenis-rust-gids-en"
main = "dist/server/server.js"
compatibility_date = "2024-09-23"
compatibility_flags = ["nodejs_compat"]

```

```

[assets]
directory = "./dist/client"

```

Notes on this configuration:

- `main = "dist/server/server.js"` is the Worker entry point — the SSR handler. This is used when deploying directly as a Worker via `wrangler deploy`. The `_worker.js` copy step in the npm scripts is a separate convention specifically for Cloudflare Pages' auto-detection of SSR handlers.
- `compatibility_flags = ["nodejs_compat"]` is required because the Nitro/TanStack Start server bundle relies on Node.js API polyfills.

- The `[assets]` block serves static assets from `./dist/client` via Cloudflare Workers Static Assets.
- Neither file defines `routes`, a custom domain, a `zone_id`, or any KV/D1/R2 bindings — domain/route attachment is not present in these config files and would need to be configured separately (e.g. in the Cloudflare dashboard).
- The EN deployment (`erfenis-rust-gids-en`) needs to be built with `npm run build:pages:en` specifically before being deployed under `wrangler.en.toml`, since that build sets `VITE_DEFAULT_LANG=en`.
- No CI/CD pipeline exists in the repository (no `.github/workflows/` directory, no `netlify.toml`, no `vercel.json`, no `Dockerfile`). Deployment is done manually (e.g. `wrangler deploy`) or via a Cloudflare Pages Git integration configured outside this repository.

6. Environment Variables

Variable	Type	Used in	Purpose
<code>VITE_DEFAULT_LANG</code>	Build-time (<code>import.meta.env</code>)	<code>src/lib/i18n.tsx</code> , <code>src/components/Header.tsx</code>	Set to "en" by <code>build:pages:en</code> (via <code>cross-env</code>); determines the default site language (nl if unset, en if set) and whether the language-toggle button is shown in the header.
<code>NODE_ENV</code>	Server runtime (<code>process.env</code>)	<code>src/lib/config.server.ts</code>	Read via a <code>getServerConfig()</code> helper; not referenced elsewhere in the codebase currently.

No `.env`, `.env.example`, or `.dev.vars` files exist anywhere in the repository. `.gitignore` lists `.dev.vars` and `*.local` as ignored patterns, but no such files are present or committed. `src/lib/config.server.ts` also contains commented-out placeholder fields (`databaseUrl`, `stripeSecretKey`) showing an intended pattern for future secrets, but no database or Stripe integration is currently wired up.

7. Third-Party Services & External Resources

- **Contact form** (`src/routes/contact.tsx`): the form's submit handler shows a toast notification (via `sonner`) and navigates the user to `/bedankt`. It does not call `fetch()` or any API, and does not use any email-sending service (no EmailJS, Formspree, or Cloudflare email binding). No form data is currently sent or stored anywhere.
- **Map**: the contact page embeds a static OpenStreetMap iframe (`https://www.openstreetmap.org/export/embed.html?bbox=...`) showing a general Netherlands view. No API key is required, and the embedded location is not a specific pinned business address.
- **Fonts**: loaded from the Google Fonts CDN (`fonts.googleapis.com` / `fonts.gstatic.com`), declared as `<link>` tags with `preconnect` hints in `src/routes/__root.tsx`. Font families used: Cormorant Garamond and Karla.
- **Analytics**: none found in the codebase (no Google Analytics, Plausible, Hotjar, or similar).
- **Chat widgets**: none found.

- **Social preview image:** the `og:image` / `twitter:image` meta tags in `src/routes/__root.tsx` point to a Lovable.dev-hosted Cloudflare R2 URL (`https://pub-bb2e103a32db4e198524a2e9ed8f35b4.r2.dev/...`).
- **Canonical URLs:** `src/routes/contact.tsx` and `src/routes/gratis-gids.tsx` set canonical URLs pointing to `https://erfenis-rust-gids.lovable.app/...` (the Lovable.dev preview domain).

8. Internationalization (i18n)

The NL/EN language handling is implemented directly in application code, not via a library (no `i18next`, no `react-intl`) and not via a CMS:

- `src/lib/translations.ts` exports a single object keyed by language (`"nl" | "en"`), containing nested strings and content objects for navigation, footer, per-page copy, and knowledge-base articles for both languages.
- `src/lib/i18n.tsx` defines a `LanguageProvider` (React Context) that determines the default language from the `VITE_DEFAULT_LANG` build-time variable, and on the client overrides that default if a language was previously stored in `localStorage` (key: `"lang"`). It exposes `useT()` (current language's translation object) and `useLang() ({ lang, setLang, t })`.
- The language toggle button is in `src/components/Header.tsx`; clicking it calls `setLang()` and persists the choice to `localStorage`.
- In practice, the NL and EN sites are two separate builds/deployments of the same codebase (`build:pages` vs. `build:pages:en`, deployed via `wrangler.toml` vs. `wrangler.en.toml`), each with a client-side toggle for switching language within whichever build the visitor is on.
- `src/routes/__root.tsx` sets `<html lang="nl">` in the root document shell regardless of which build or selected language is active.

9. Static Assets & Content

- All images are stored under `src/assets/` (hero images, team photos, testimonials, and the logo `erfeniswijzer-logo.jpeg`) and imported directly as ES modules in route/component files (e.g. `import heroImg from "@assets/kennisbank-hero.jpg"`). Vite bundles, hashes, and optimizes these into `dist/client/assets/` at build time.
- There is no `public/` directory, so there is no separate static-file-serving convention outside of Vite's asset pipeline.
- There is no CMS, database, or external content API. All page content (navigation labels, headings, body copy, knowledge-base articles, testimonials, CTAs) is hardcoded either directly in route components (`src/routes/*.tsx`) or centrally in `src/lib/translations.ts`. Content changes require a code change and redeploy.
- `src/lib/api/example.functions.ts` is a boilerplate TanStack Start server-function example and is not an active content source.

10. Package Manager

Both `package-lock.json` and `bun.lock` are present and tracked in git, alongside a `bunfig.toml` (which configures a 24-hour supply-chain install guard, with explicit excludes for `@lovable.dev/*` packages). No

`packageManager` field is set in `package.json`.

11. Git Repository Info

- Remote origin: `https://github.com/AmreshBandhoe/erfenis-rust-gids.git`
- Branches: only `main` exists, both locally and on the remote.
- `wrangler.en.toml` is present on disk but is currently untracked in git (not committed).
- No `README.md` exists at the repository root. The only markdown documentation present (besides this file) is `src/routes/README.md` (a short note on TanStack Start's file-based routing conventions) and `.lovable/plan.md` (Lovable's internal build/design plan, containing design-system notes such as colors and fonts).